

COMP 3311 数据库管理系统 — 课程总结 (Part 3: Lectures 12–15 — NoSQL & MongoDB)

课程: COMP 3311 Database Management Systems
时间: 2026年7月3日–11日
覆盖: Lecture 12 – Lecture 15 (NoSQL 数据库系统 & MongoDB)
配套 Part 2: Lectures 8–11 (SQL DML & DDL)

MongoDB 示例数据库 (Bank Schema) :

```
clients: {clientId, name, hkid, address, district, rating,
  accounts: [{accountNo, balance, branch}],
  loans: [{loanNo, amount, year, branch}],
  tags: [...]}
branches: {branch, district, liabilities, assets,
  accounts: [...], loans: [...]}
```

目录

- 1. Lecture 12: NoSQL 数据库管理系统
- 2. Lecture 13: MongoDB — 数据模型与查询语言
- 3. Lecture 14: MongoDB — 数组查询与模式验证
- 4. Lecture 15: MongoDB — 聚合框架

12. Lecture 12: NoSQL 数据库管理系统

12.1 NoSQL 的动机：大数据 (Big Data)

大数据的三个维度 (3V)：

维度	含义
Volume (容量)	数据量极大，持续增长
Velocity (速度)	数据产生和数据分析速度极快
Variety (多样性)	非结构化、半结构化数据，超越关系型

- 许多应用愿意牺牲关系型 DBMS 特性以获得高可扩展性

12.2 NoSQL 运动核心思想

RDBMS (关系型)	NoSQL
强调数据 一致性	接受 最终一致性
正式模式、类型、参照完整性、事务	灵活模式或无模式
垂直扩展 为主 (增加单机能力)	水平扩展 (增加节点数)
水平扩展时协调开销大	近线性的水平扩展 + 高可用性
丰富查询功能 (SQL)	简单查询 / API

最终一致性 (Eventual Consistency): 数据及其副本在事务后的某个时间点最终达到一致, 不保证瞬时一致性。

12.3 四种 NoSQL 类型

类型	数据组织	代表系统
Key-Value	唯一键 → 无模式值 (字节串)	BerkeleyDB, Redis, Memcached, Riak
Document	键 → 半结构化文档 (JSON/BSON/XML)	MongoDB, CouchDB, OrientDB
Columnar	按列组织 (非按行), null 不占空间	Cassandra, HBase, Amazon SimpleDB
Graph	节点 + 边, 支持图遍历查询	Neo4J, FlockDB, OrientDB

Key-Value DBMS

- 基本操作: `put(key, value)`, `get(key)`, `delete(key)`
- 键是**唯一搜索标准**
- 值通常为无模式字节, DBMS 不解释其内容

Document DBMS

- 一种 Key-Value DBMS, 值是 JSON-like 键值对集合
- 支持**非键属性查询**
- JSON 最广泛使用

Columnar DBMS

- 按**列**而非**行**组织数据

- null 不占存储空间
- 可直接按列值筛选（无需索引）

Graph DBMS

- N:M 关系自然映射为图结构，不需要连接表
- 查询使用图模式匹配（如 Cypher 语言）

12.4 JSON 语法

元素	语法	说明
对象 Object	{ }	无序键值对集合，键必须用双引号
数组 Array	[]	无序值集合，逗号分隔
字符串 String	" "	双引号，反斜杠转义
数字 Number	—	整数、实数、科学记数法（不能有多余零）
布尔 Boolean	true / false	—
Null	null	—

12.5 Oracle SQL/JSON 函数

函数	功能	默认 null 处理
json_object	从查询元组构造 JSON 对象集合 （每个元组一个对象）	null on null
json_array	从查询元组构造 JSON 数组集合 （每个元组一个数组）	absent on null
json_objectagg	聚合多行为 单个 JSON 对象	null on null
json_arrayagg	聚合多行为 单个 JSON 数组	absent on null

可选子句：

- null on null / absent on null — 控制 SQL null 的处理
- returning clause — 指定返回类型（默认 varchar2(4000)）
- strict — 检查输出是否为合法 JSON
- with unique keys — 确保 JSON 键无重复

12.6 RDBMS vs NoSQL 总结

特性	RDBMS	NoSQL
数据组织	关系表	Key-value / Document / Column / Graph
分布	通常单服务器	多分布式服务器
扩展性	垂直扩展，难以水平扩展	易于水平扩展，易于复制
模式	模式驱动	无模式 / 灵活模式
查询语言	SQL	简单 API 或专用语言
功能集	丰富（触发器、视图、存储过程等）	简单 API
数据量	正常规模	海量数据 / 极高读写频率

13. Lecture 13: MongoDB — 数据模型与查询语言

13.1 MongoDB 数据模型

数据库 (Database)

- └─ 集合 (Collection) - 类似于关系表
 - └─ 文档 (Document) - 类似于行, JSON-like (BSON)
 - └─ 字段 (Field) - 键值对

核心概念：

概念	关系型	MongoDB
表	Table	Collection
行	Row	Document
列	Column	Field
主键	Primary Key	_id (自动生成或用户指定)
模式	Schema-driven	Schemaless (灵活模式)

MongoDB 额外数据类型： Date, BinData, Regular Expression, Object ID

13.2 MongoDB 数据库设计

关系表示方式

方式	说明	优点	缺点
References (引用)	文档中包含其他文档的 ID 链接	数据规范化, 适合 N:M 关系	需要多次查询
Embedded Data (嵌入式)	相关数据存储在同一个文档中	单次操作即可读写, 读性能好	数据重复 (反规范化)

选择依据: 取决于应用的最常见查询模式。

设计关键: 集合中文档的字段可以不同, 字段的数据类型在文档间也可以不同 (但有模式验证可强制一致性)。

13.3 聚合表达式

类别	特征	示例
字段路径	\$ 前缀 + 字段路径	'\$district', '\$accounts.balance'
操作符	\$ 前缀	\$cond, \$sum, \$avg
变量	\$\$ 前缀	\$\$REMOVE, \$\$this

13.4 查询文档

```
db.collection.find(queryFilter, projection)
db.collection.findOne(queryFilter, projection)
```

- `find` 返回所有匹配文档; `findOne` 返回一个匹配文档 (不确定返回哪一个)
- `queryFilter` 为空则返回所有文档

13.5 投影 (Projection)

规则	说明
值 = 1 (或 true)	包含 该字段
值 = 0 (或 false)	排除 该字段
默认	<code>_id</code> 始终包含 (除非显式排除)
互斥规则	除 <code>_id</code> 外, 包含和排除 不能同时指定
新增/重命名字段	新字段名: '\$existingField'

13.6 比较表达式操作符

操作符	含义
<code>\$eq</code>	等于（默认，可省略）
<code>\$ne</code>	不等于
<code>\$lt</code> / <code>\$lte</code>	小于 / 小于等于
<code>\$gt</code> / <code>\$gte</code>	大于 / 大于等于
<code>\$cmp</code>	比较两个值（返回 -1, 0, 1）

13.7 布尔表达式操作符

操作符	语法	说明
<code>\$and</code>	<code>{ \$and: [{p1}, {p2}, ...] }</code>	逻辑与（默认行为）
<code>\$or</code>	<code>{ \$or: [{p1}, {p2}, ...] }</code>	逻辑或
<code>\$nor</code>	<code>{ \$nor: [{p1}, {p2}, ...] }</code>	逻辑非或（所有条件都不满足）
<code>\$not</code>	<code>{ \$not: {predicate} }</code>	逻辑非

重要陷阱：

- 默认 AND 对**同名字段**的多个条件：第二个条件**覆盖**第一个条件
 - ✗ `{rating: { $gte: 7 }, rating: { $lte: 9 }}` → 只应用了 `$lte: 9`
 - ✓ `{ $and: [{rating: { $gte: 7 }}, {rating: { $lte: 9 }}] }`
 - ✓ `{rating: { $gte: 7, $lte: 9 }}` — 同名字段可用数组写法

13.8 条件表达式操作符

操作符	用途
<code>\$ifNull</code>	返回第一个非 null 表达式值 / 替代值
<code>\$cond</code>	if-then-else；返回 trueCase / falseCase
<code>\$switch</code>	多分支条件判断（branches 数组）

`$cond` 语法：

```
{ $cond: { if: booleanExpr, then: trueCase, else: falseCase }}  
// 或数组语法  
{ $cond: [booleanExpr, trueCase, falseCase] }
```

13.9 查询谓词操作符 `$expr`

- 允许在查询条件中使用**聚合表达式**
- 用于**比较同一文档的两个字段**或字段计算后比较

```
// 资产 < 负债的分行  
db.branches.find({ $expr: { $lt: ['$assets', '$liabilities'] } })
```

13.10 正则表达式 `$regex`

```
// 名字以 Steven 或 Stephen 开头  
db.clients.find({ name: { $regex: /^Ste(v|ph)en/ } })
```

- 选项: `'i'` 不区分大小写

13.11 Null/缺失 比较

查询	匹配条件
<code>{field: null}</code>	null 或 字段不存在
<code>{field: { \$ne: null }}</code>	字段存在且非 null
<code>{field: { \$exists: true }}</code>	字段存在 (含 null)
<code>{field: { \$exists: false }}</code>	字段不存在

14. Lecture 14: MongoDB — 数组查询与模式验证

14.1 查询数组元素

访问方式	语法	说明
按位置	<code>'loans.0.amount'</code>	访问数组第 1 个元素的 amount 字段
按字段名	<code>'loans.amount'</code>	数组中至少一个元素满足条件

关键警告：

- 对数组字段的 AND 条件可由不同数组元素满足（不是同一元素的 AND）
- 解决方案：使用 `$elemMatch`

14.2 数组查询操作符

操作符	功能
<code>\$elemMatch</code>	至少一个数组元素同时满足所有条件
<code>\$all</code>	数组包含所有指定值（任意顺序）
<code>\$size</code>	数组长度等于指定数字
<code>\$in / \$nin</code>	字段值在 / 不在指定数组中
<code>\$slice</code>	返回数组的前 / 后 N 个元素或跳过 N 个后返回 M 个

`$elemMatch` 在投影中：只返回第一个匹配条件的数组元素。

14.3 数组表达式操作符

操作符	功能
<code>\$size</code>	返回数组元素个数
<code>\$isArray</code>	判断是否为数组
<code>\$arrayElemAt</code>	返回指定索引位置的元素（支持负索引）
<code>\$first / \$last</code>	返回数组第一个 / 最后一个元素
<code>\$firstN / \$lastN</code>	返回数组前 / 后 N 个元素
<code>\$minN / \$maxN</code>	返回数组最小 / 最大的 N 个值
<code>\$filter</code>	筛选数组中满足条件的元素
<code>\$map</code>	对数组每个元素应用表达式
<code>\$reduce</code>	将数组元素合并为单个值
<code>\$concatArrays</code>	拼接多个数组
<code>\$sortBy</code>	对数组元素排序

14.4 集合表达式操作符

操作符	功能
<code>\$setDifference</code>	只在 array1 中的唯一元素
<code>\$setIntersection</code>	所有数组共有唯一元素
<code>\$setUnion</code>	所有数组中的唯一元素
<code>\$setEquals</code>	是否有相同不同元素
<code>\$setIsSubset</code>	array1 是否为 array2 子集
<code>\$allElementsTrue</code>	是否无元素为 false
<code>\$anyElementTrue</code>	是否有元素为 true

14.5 去重、计数、排序、限制

方法	功能
<code>distinct(field)</code>	返回指定字段的唯一值（作为数组）
<code>count()</code>	返回查询结果文档数量
<code>sort({field: 1/-1})</code>	按字段排序（1=升序, -1=降序）
<code>limit(n)</code>	限制结果为 n 个文档
<code>skip(n)</code>	跳过前 n 个文档

Sort 注意事项：

- 排序只有在**存在**于输入文档中的字段上进行（不能在投影的计算字段上排序）
- sort 总是在 projection 之前执行
- null/缺失字段值视为**最小值**

14.6 JSON Schema 验证

MongoDB 使用 `$jsonSchema` 定义文档验证规则：

关键字	含义
<code>bsonType</code>	BSON 数据类型（如 <code>'int'</code> , <code>'string'</code> , <code>'array'</code> , <code>'object'</code> ）
<code>required</code>	必须存在的字段数组
<code>properties</code>	每个字段的模式定义
<code>enum</code>	字符串值枚举列表
<code>minimum</code> / <code>maximum</code>	数值范围

关键字	含义
<code>minLength</code> / <code>maxLength</code>	字符串/数组长度限制
<code>pattern</code>	正则表达式匹配
<code>description</code>	验证失败时的错误提示
<code>uniqueItems</code>	数组元素必须唯一

15. Lecture 15: MongoDB — 聚合框架

15.1 聚合框架概念

聚合框架 = **数据处理流水线**：文档流经多个阶段，每个阶段执行一个操作。

```
input documents → stage 1 → stage 2 → ... → stage n → output documents
```

- 每个阶段的输入和输出都是**文档**
- 聚合方法：`db.collection.aggregate([pipeline])`

15.2 常用聚合阶段

阶段	功能	SQL 对应
<code>\$project</code>	限制/变换字段	SELECT
<code>\$match</code>	过滤文档	WHERE/HAVING
<code>\$group</code>	按标识符分组，应用累加器	GROUP BY
<code>\$count</code>	计算文档数	COUNT
<code>\$sort</code>	排序	ORDER BY
<code>\$limit</code>	限制输出文档数	FETCH
<code>\$skip</code>	跳过前 N 个文档	OFFSET
<code>\$unwind</code>	数组展开（一个元素 → 一个文档）	N/A
<code>\$lookup</code>	左外连接	LEFT JOIN

15.3 累加器 (Accumulators)

累加器	功能
<code>\$count</code>	计数

累加器	功能
\$sum	求和
\$avg	平均值
\$max / \$min	最大 / 最小值
\$median	近似中位数
\$stdDevPop	总体标准差
\$addToSet	添加值到数组（无重复）
\$push	追加值到数组（允许重复）

- 在 \$project 中：对**单个文档**的数组字段计算
- 在 \$group 中：对**多个文档**跨文档计算

15.4 \$project 阶段

```
{ $project: { _id: 0, district: 1 } }
```

- 输出仅含指定字段（与 find 投影相同）
- 但不能是空文档

15.5 \$match 阶段

```
{ $match: { district: 'Islands' } }
```

- 与 find 的 queryFilter 相同
- **一般不允许聚合表达式**（\$expr 例外）

15.6 \$sort、\$limit、\$skip 阶段

与同名方法功能相同，但：

- **不能使用聚合表达式**
- 与 find 中的方法不同：执行顺序**由阶段顺序决定**（而非强制 sort → skip → limit）

15.7 \$group 阶段

```
{ $group: {
  _id: groupKey, // 分组键：null → 全局聚合
```

```
field: {accumulator: expr}
}}
```

- 每个唯一分组键输出一个文档
- 复合键需用 `{field1: '$field1', field2: '$field2'}` 格式

15.8 \$count 阶段

```
{ $count: 'fieldName' }
```

- 计算到达该阶段的文档数
- 输出一个文档，其中 `fieldName` 为计数值

15.9 \$unwind 阶段 —— 关键！

将数组字段**展开**为多个文档（每个数组元素一个文档）。

```
{ $unwind: { path: '$accounts' } }
// 或保留 null/空数组的文档：
{ $unwind: { path: '$accounts', preserveNullAndEmptyArrays: true } }
```

为什么需要 unwind？ — 累加器无法直接应用于数组字段。

\$unwind vs \$match 顺序的重要性：

- `$unwind` → `$match`：在每个展开的数组元素上过滤 → **正确**
- `$match` → `$unwind`：匹配整个文档（数组包含至少一个匹配元素） → 可能包含不想要的元素

15.10 \$addToSet — 去重

```
{ $group: {
  _id: null,
  uniqueAccounts: { $addToSet: { accountNo: '$accounts.accountNo', ... } }
}}
```

- 保证数组中没有重复元素
- 常用于消除重复（如同一账户被多个客户持有）

15.11 \$push — 保留字段

```
{ $group: {  
  _id: '$rating',  
  clients: { $push: { clientId: '$clientId', name: '$name' } }  
}}
```

- 在分组时保留输入文档的字段信息
- \$each 修饰符可一次添加多个值
- \$slice、\$sort、\$position 修饰符可用（需配合 \$each）

15.12 \$lookup — 左外连接

```
{ $lookup: {  
  from: 'branches', // 被连接的集合  
  localField: 'accounts.branch', // 输入文档的字段  
  foreignField: 'branch', // 被连接集合的字段  
  as: 'branchInfo' // 输出数组字段名  
}}
```

- 为每个输入文档**添加一个数组字段**，包含匹配的文档
- 相当于 SQL 的 LEFT OUTER JOIN

15.13 聚合框架流水线设计模式

计算去重的跨分支总额：

1. \$unwind 展开数组
2. \$group + \$addToSet 去重
3. \$unwind 再次展开去重后的数组
4. \$group + \$sum 聚合

找到每组的最大值获得者：

1. \$unwind → \$group (+ \$push 保留信息 + \$max 记录最大值)
2. \$unwind → \$match (用 \$expr 比较)

模拟 Top-N（含并列）：

1. \$group + \$push → \$sort → \$skip → \$limit

NoSQL & MongoDB 核心知识地图

NoSQL 基础 (L12)

- └─ 大数据 3V: Volume, Velocity, Variety
- └─ NoSQL 核心理念: 最终一致性、水平扩展、灵活模式
- └─ 四种类型: Key-Value, Document, Columnar, Graph
- └─ JSON 语法: 对象 {}, 数组 [], 六种数据类型
- └─ Oracle SQL/JSON 函数: json_object, json_array, json_objectagg, json_arrayagg

MongoDB 数据模型 (L13)

- └─ 数据库 → 集合 → 文档 → 字段
- └─ _id (自动或用户定义, 唯一)
- └─ References vs Embedded Data
- └─ 聚合表达式: \$字段路径, \$操作符, \$\$变量
- └─ 基本查询: find/findOne + queryFilter + projection

MongoDB 查询语言 (L13-L14)

- └─ 比较: \$eq, \$ne, \$lt, \$lte, \$gt, \$gte, \$cmp
- └─ 布尔: \$and, \$or, \$nor, \$not
- └─ 条件: \$ifNull, \$cond, \$switch
- └─ 谓词: \$expr (跨字段比较)
- └─ 正则: \$regex
- └─ Null/缺失: null, \$exists, \$ne: null
- └─ 数组查询: 位置访问, \$elemMatch, \$all, \$size
- └─ 数组表达式: \$filter, \$map, \$reduce, \$sortBy
- └─ 集合操作: \$setDifference, \$setIntersection, \$setUnion
- └─ 结果处理: distinct, count, sort, limit, skip
- └─ 模式验证: \$jsonSchema + bsonType

MongoDB 聚合框架 (L15)

- └─ 流水线概念: 阶段序列处理文档
- └─ \$project: 限制/变换字段
- └─ \$match: 过滤文档
- └─ \$group: 分组聚合 (_id + 累加器)
- └─ \$count: 计数
- └─ \$sort / \$limit / \$skip: 排序和限制
- └─ \$unwind: 数组展开 (核心!)
- └─ \$addToSet: 去重收集
- └─ \$push: 保留字段信息
- └─ \$lookup: 左外连接
- └─ 累加器: \$sum, \$avg, \$max, \$min, \$median, \$addToSet, \$push

课程参考: "Fundamentals of Database Systems" (教材章节标注于各 Lecture 幻灯中)

关联文件: [COMP3311_Lecture_Summary_Part1](#) — E-R 模型、关系设计、范式化、关系代数 | [COMP3311_Lecture_Summary_Part2](#) — SQL DML & DDL